



Laboratorio 3

Prueba de Servicios REST con HATEOAS, Docker, MySQL y Kubernetes

Información	Detalle
Universidad	Universidad de Antioquia
Curso	Arquitectura de Software
Laboratorio	Laboratorio 3
Proyecto	vuelokbt
Tema	API RESTful para gestión de vuelos con Spring Boot, MySQL, Docker, Kubernetes y ArgoCD
Profesor	Diego José Luis Botia V.
Integrantes	Santiago Palacio Cárdenas, Sarai Restrepo Rodríguez, Juan Pablo Herrera Jaramillo, Jimena Muñoz Gómez
Repositorio	https://github.com/spalacioc05/lab3-20261.git
Fecha	22 de mayo de 2026

Índice

Laboratorio 3.....	1
Prueba de Servicios REST con HATEOAS, Docker, MySQL y Kubernetes.....	1
Índice.....	2
Introducción.....	3
Objetivos.....	4
Objetivo general.....	4
Objetivos específicos.....	4
Herramientas de software empleadas.....	5
Arquitectura propuesta.....	7
Visión general.....	7
Componentes principales.....	7
Arquitectura por capas.....	8
Diagrama general de arquitectura.....	8
Descripción del flujo.....	9
Procedimiento.....	10
1. Revisión inicial del proyecto.....	10
2. Configuración de la aplicación.....	12
3. Implementación de la API REST.....	13
4. Modelo de datos.....	15
5. Documentación con Swagger/OpenAPI.....	17
6. Construcción del proyecto con Maven.....	19
7. Contenerización con Docker.....	20
8. Despliegue en Kubernetes con Minikube.....	22
9. Verificación de MySQL en Kubernetes.....	25
10. Sincronización GitOps con ArgoCD.....	26
11. Escalabilidad horizontal.....	30
12. Pruebas funcionales.....	33
Crear vuelo.....	33
Listar vuelos.....	35
Consultar vuelo por ID.....	35
Consultar mejores vuelos.....	36
Actualizar vuelo.....	36
Conclusiones.....	38
Bibliografía.....	39
Proyecto anexo en GitHub.....	40

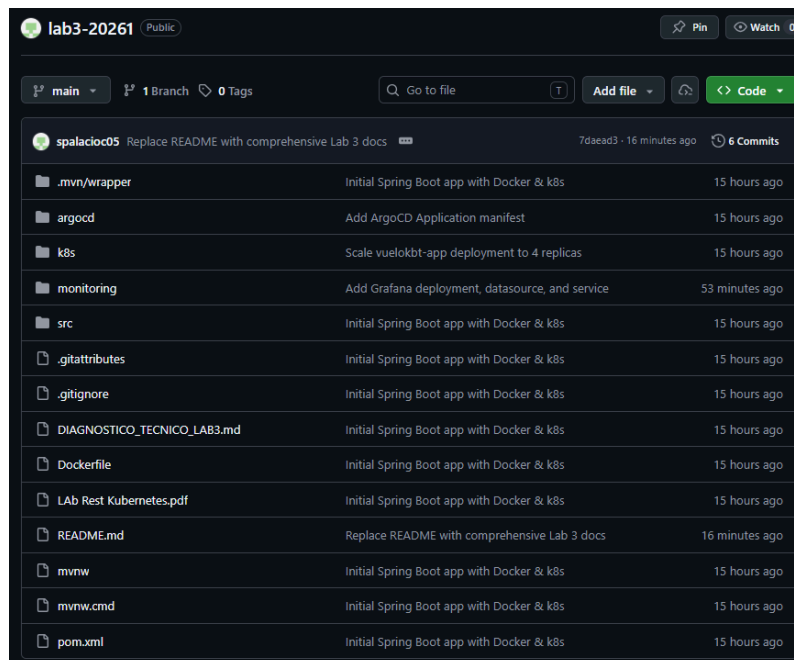
Introducción

El presente informe documenta el desarrollo, despliegue y validación del **Laboratorio 3 de Arquitectura de Software**, cuyo objetivo principal fue construir una aplicación RESTful para la gestión de vuelos, integrando varias tecnologías usadas en escenarios reales de desarrollo y despliegue de software.

El proyecto trabajado se denomina vuelokbt y fue desarrollado con **Spring Boot 2.7.17** y **Java 11**. La aplicación expone una API REST para crear, consultar, actualizar, eliminar y listar vuelos destacados según criterios definidos en el dominio. Además, utiliza **MySQL** como motor de persistencia, **Maven** como herramienta de construcción, **Swagger/OpenAPI** para documentación de la API, **Docker** para contenerización, **Kubernetes/Minikube** para despliegue local en clúster y **ArgoCD** como mecanismo de sincronización GitOps.

Más allá de implementar un CRUD básico, el laboratorio permitió comprender cómo una aplicación backend pasa de ejecutarse localmente a estar empaquetada en una imagen Docker y desplegada sobre una infraestructura orquestada. También permitió reconocer la importancia de configurar correctamente variables de entorno, servicios de base de datos, manifiestos de Kubernetes, exposición de servicios, documentación de endpoints y evidencias de ejecución.

Durante el desarrollo se identificaron y corrigieron varios puntos importantes: configuración inicial acoplada a MySQL local, necesidad de parametrizar variables, ausencia inicial de archivos de despliegue, manejo REST de la aplicación, documentación del flujo y preparación de evidencias para demostrar el funcionamiento. El resultado final es una solución organizada que integra backend, persistencia, documentación, contenedores y despliegue declarativo.



Objetivos

Objetivo general

Diseñar, implementar, contenerizar y desplegar una API RESTful para la gestión de vuelos, utilizando Spring Boot, MySQL, Docker, Kubernetes/Minikube y ArgoCD, con el fin de aplicar conceptos de arquitectura de software, despliegue de servicios, documentación de APIs y operación básica en entornos contenerizados.

Objetivos específicos

- Implementar una API REST en Spring Boot para administrar vuelos.
- Integrar una base de datos MySQL para persistir la información de los vuelos.
- Documentar la API mediante Swagger/OpenAPI.
- Preparar el proyecto para ejecución local y empaquetamiento mediante Maven.
- Crear una imagen Docker de la aplicación a partir del artefacto `.jar`.
- Desplegar la aplicación y la base de datos en Kubernetes usando Minikube.
- Configurar Services y Deployments para exponer la aplicación y conectar con MySQL dentro del clúster.
- Implementar una estrategia GitOps básica mediante ArgoCD, apuntando a los manifiestos del repositorio.
- Aplicar escalabilidad horizontal aumentando el número de réplicas del Deployment.
- Documentar comandos, evidencias, capturas y conclusiones del proceso realizado.

Herramientas de software empleadas

Para el desarrollo del laboratorio se utilizaron herramientas orientadas a construcción backend, persistencia, documentación, contenerización, despliegue y operación en clúster.

Herramienta / Tecnología	Uso dentro del laboratorio
Java 11	Lenguaje principal de desarrollo de la aplicación.
Spring Boot 2.7.17	Framework usado para construir la API REST.
Maven / Maven Wrapper	Construcción, gestión de dependencias y empaquetamiento del proyecto.
Spring Web	Exposición de endpoints REST.
Spring Data JPA	Acceso a datos y persistencia de entidades.
MySQL	Base de datos relacional para almacenar los vuelos.
Springdoc OpenAPI	Generación de documentación Swagger/OpenAPI.
Spring HATEOAS / HAL Explorer	Soporte para principios HATEOAS y exploración HAL.
Docker	Contenerización de la aplicación.
Kubernetes	Orquestación de la aplicación y la base de datos.
Minikube	Clúster local de Kubernetes para pruebas.
ArgoCD	Sincronización GitOps de los manifiestos Kubernetes.
Grafana	Monitoreo ligero opcional dentro del laboratorio.
Git / GitHub	Control de versiones y repositorio del proyecto.
Postman / Swagger	Pruebas funcionales de los endpoints.

Files

main

+

Q

Go to file

T

>

.mvn

>

argocd

>

k8s

>

monitoring

>

src

.gitattributes

.gitignore

Dockerfile

Lab Rest Kubernetes.pdf

README.md

mvnw

mvnw.cmd

pom.xml

lab3-20261 / pom.xml

...

spalacioc05

Initial Spring Boot app with Docker & k8s

08ebbf2 · 16 hours ago

History

Code

Blame

118 lines (111 loc) · 3.4 KB

↗

🔍

🔄

Raw

📄

📥

✎

⌵

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
4   <modelVersion>4.0.0</modelVersion>
5   <parent>
6     <groupId>org.springframework.boot</groupId>
7     <artifactId>spring-boot-starter-parent</artifactId>
8     <version>2.7.17</version>
9     <relativePath/> <!-- lookup parent from repository -->
10  </parent>
11  <groupId>com.udea</groupId>
12  <artifactId>vuelokt</artifactId>
13  <version>0.0.1-SNAPSHOT</version>
14
15  <properties>
16    <java.version>11</java.version>
17  </properties>
18  <dependencies>
19    <dependency>
20      <groupId>org.springframework.boot</groupId>
21      <artifactId>spring-boot-starter-actuator</artifactId>
22    </dependency>
23  </dependencies>
```

Files

main

+

Q

Go to file

T

>

.mvn

>

argocd

>

k8s

>

monitoring

>

src

.gitattributes

.gitignore

Dockerfile

Lab Rest Kubernetes.pdf

README.md

mvnw

mvnw.cmd

pom.xml

lab3-20261 / pom.xml

↑ Top

Code

Blame

118 lines (111 loc) · 3.4 KB

↗

🔍

🔄

Raw

📄

📥

✎

⌵

```
23 <dependency>
24   <groupId>org.springframework.boot</groupId>
25   <artifactId>spring-boot-starter-data-jpa</artifactId>
26 </dependency>
27 <dependency>
28   <groupId>org.springframework.boot</groupId>
29   <artifactId>spring-boot-starter-data-rest</artifactId>
30 </dependency>
31 <dependency>
32   <groupId>org.springframework.boot</groupId>
33   <artifactId>spring-boot-starter-hateoas</artifactId>
34 </dependency>
35 <dependency>
36   <groupId>org.springframework.boot</groupId>
37   <artifactId>spring-boot-starter-validation</artifactId>
38 </dependency>
39 <dependency>
40   <groupId>org.springframework.boot</groupId>
41   <artifactId>spring-boot-starter-web</artifactId>
42 </dependency>
43 <dependency>
44   <groupId>org.springframework.data</groupId>
45   <artifactId>spring-data-rest-hal-explorer</artifactId>
46 </dependency>
47
48 <dependency>
49   <groupId>org.springframework.boot</groupId>
50   <artifactId>spring-boot-configuration-processor</artifactId>
51   <optional>true</optional>
52 </dependency>
53
54 <dependency>
```

Arquitectura propuesta

Visión general

La arquitectura propuesta corresponde a una aplicación backend en capas, desplegada como servicio contenerizado y conectada a una base de datos MySQL. La aplicación sigue una estructura tradicional de Spring Boot, separando responsabilidades entre controlador, servicio, repositorio y modelo de datos.

A nivel de infraestructura, el backend se empaqueta como imagen Docker y se despliega en Kubernetes mediante un Deployment. La base de datos MySQL también se despliega dentro del clúster mediante sus propios manifiestos. Ambos componentes se comunican internamente usando Services de Kubernetes.

Adicionalmente, el repositorio incluye configuración para ArgoCD, de modo que los manifiestos ubicados en la carpeta k8s/ puedan sincronizarse de forma declarativa desde GitHub hacia el clúster.

Componentes principales

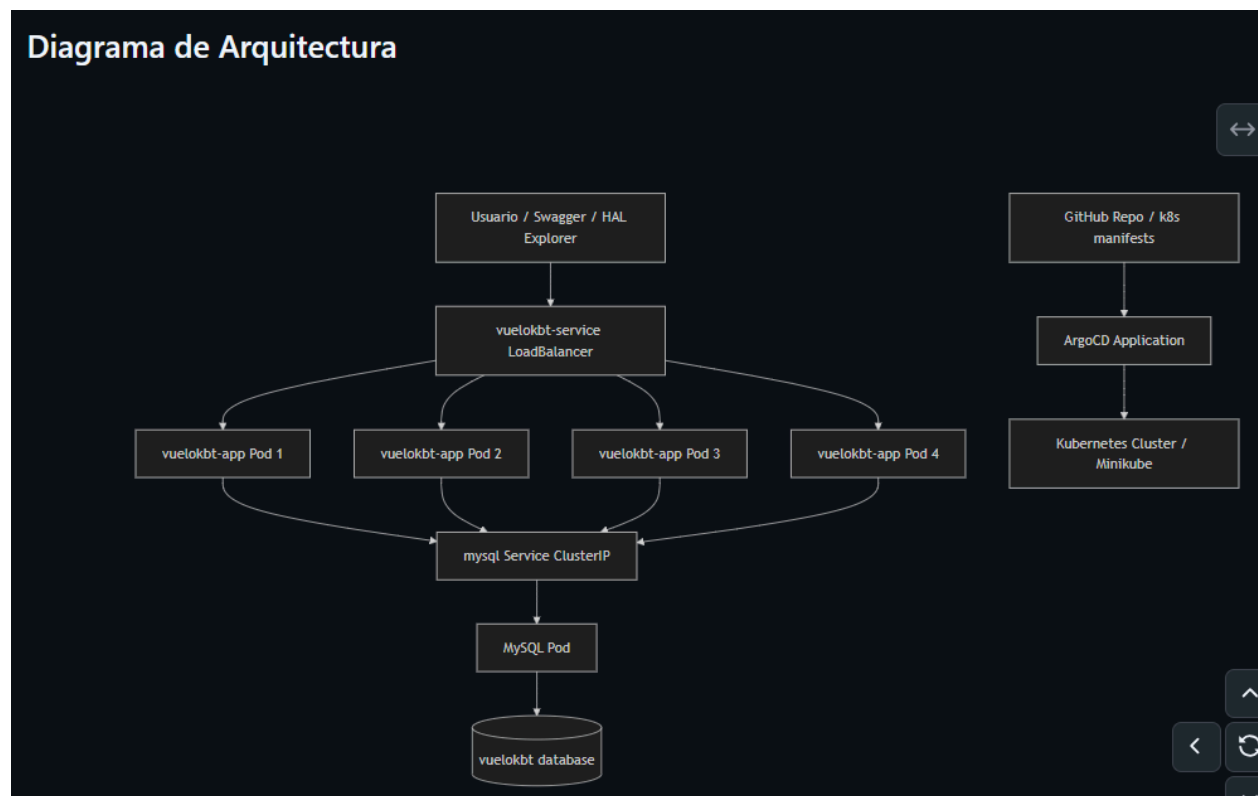
Componente	Responsabilidad
Usuario / cliente API	Consume la API mediante Swagger, Postman o navegador.
FlightController	Expone los endpoints REST bajo la ruta base /flight.
FlightService	Contiene la lógica de negocio para guardar, consultar, actualizar y eliminar vuelos.
IFlightDAO	Repositorio Spring Data JPA para operaciones sobre la entidad Flight.
Flight	Entidad JPA que representa la información persistida del vuelo.
MySQL	Base de datos relacional donde se almacenan los registros de vuelos.
Docker	Empaqueta la aplicación en una imagen ejecutable.
Kubernetes / Minikube	Ejecuta la aplicación y MySQL en pods administrados por Deployments.
ArgoCD	Sincroniza los manifiestos Kubernetes desde GitHub.
Swagger / OpenAPI	Permite consultar y probar la documentación de la API.

Arquitectura por capas

La aplicación se organiza en los siguientes paquetes principales:

Capa / paquete	Descripción
controller	Define los endpoints REST que recibe el usuario o cliente HTTP.
service	Implementa la lógica de negocio y coordina las operaciones con el repositorio.
dao	Contiene el repositorio IFlightDAO, encargado del acceso a datos.
model	Define la entidad Flight, que se persiste en MySQL.
exception	Contiene excepciones de dominio y manejo de errores asociados a la API.
OpenApiConfig	Configura información general de Swagger/OpenAPI.
VuelokbtApplication	Clase principal de arranque de la aplicación Spring Boot.

Diagrama general de arquitectura



Descripción del flujo

El usuario consume la aplicación a través del servicio expuesto por Kubernetes. Este servicio balancea el tráfico hacia las réplicas activas del backend vuelokbt-app. Cada pod de aplicación ejecuta la API Spring Boot y se conecta internamente al Service de MySQL. La base de datos se ejecuta en un pod separado y conserva la información de los vuelos.

El repositorio GitHub contiene el código fuente, el Dockerfile y los manifiestos Kubernetes. Con ArgoCD, el clúster puede sincronizar los recursos declarados en el repositorio, permitiendo un flujo GitOps en el que la infraestructura se administra desde archivos versionados.

Procedimiento

1. Revisión inicial del proyecto

El primer paso fue revisar la estructura del repositorio y los archivos principales de la aplicación. Se identificó que el proyecto ya contaba con una base Spring Boot organizada en paquetes de controlador, servicio, repositorio, modelo y excepciones.

La entidad principal del sistema es Flight, que representa un vuelo. Esta entidad contiene información como nombre del avión, número de vuelo, origen, destino, capacidad, rating, plan de vuelo y estado de cumplimiento.

Archivo / carpeta	Descripción
pom.xml	Define dependencias y configuración Maven.
src/main/java/com/udea/vuelokbt/controller	Controladores REST.
src/main/java/com/udea/vuelokbt/service	Servicios de aplicación.
src/main/java/com/udea/vuelokbt/dao	Repositorio JPA.
src/main/java/com/udea/vuelokbt/model	Entidad Flight.
src/main/resources/application.properties	Configuración de aplicación, puerto y base de datos.
Dockerfile	Construcción de la imagen Docker.
k8s/	Manifiestos Kubernetes.
argocd/	Configuración GitOps con ArgoCD.
monitoring/	Configuración opcional de Grafana.

```
lab3-20261/
├─ argocd/
│   └─ application.yml
├─ k8s/
│   ├── app-deployment.yml
│   ├── mysql-configmap.yml
│   └─ mysql-deployment.yml
├─ monitoring/
│   └─ grafana-simple.yml
├─ src/
│   ├── main/
│   │   ├── java/com/udea/vuelokbt/
│   │   │   ├── controller/
│   │   │   ├── dao/
│   │   │   ├── exception/
│   │   │   ├── model/
│   │   │   ├── service/
│   │   │   ├── OpenApiConfig.java
│   │   │   └─ VuelokbtApplication.java
│   │   └─ resources/
│   │       └─ application.properties
│   └─ test/
│       └─ java/com/udea/vuelokbt/
├─ Dockerfile
├─ pom.xml
└─ README.md
```

2. Configuración de la aplicación

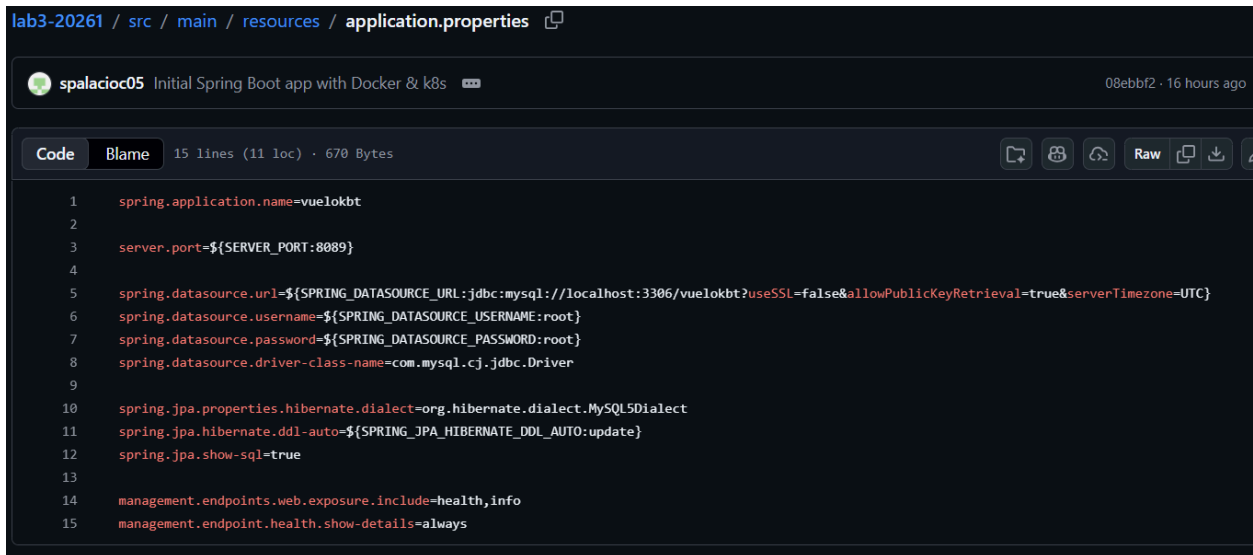
La aplicación se configuró para ejecutarse por defecto en el puerto **8089**. La conexión a base de datos se parametrizó mediante variables de entorno, lo que permite usar distintos valores en ejecución local, Docker o Kubernetes.

Configuración relevante:

```
server.port=${SERVER_PORT:8089}
spring.datasource.url=${SPRING_DATASOURCE_URL:jdbc:mysql://localhost:3306/vuelokbt?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC}
spring.datasource.username=${SPRING_DATASOURCE_USERNAME:root}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD:root}
spring.jpa.hibernate.ddl-auto=${SPRING_JPA_HIBERNATE_DDL_AUTO:update}
management.endpoints.web.exposure.include=health,info
```

Esta parametrización evita depender exclusivamente de valores fijos dentro del código fuente. En Kubernetes, por ejemplo, la aplicación se conecta al Service de MySQL usando una URL interna como:

```
jdbc:mysql://mysql:3306/vuelokbt
```



The screenshot shows a GitHub repository interface. At the top, the repository name is 'lab3-20261' and the file path is 'src / main / resources / application.properties'. The file is owned by 'spalacioc05' and was last updated '08ebb12 · 16 hours ago'. The file is 15 lines long, 11 lines of code, and 670 bytes. The code is as follows:

```
1  spring.application.name=vuelokbt
2
3  server.port=${SERVER_PORT:8089}
4
5  spring.datasource.url=${SPRING_DATASOURCE_URL:jdbc:mysql://localhost:3306/vuelokbt?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC}
6  spring.datasource.username=${SPRING_DATASOURCE_USERNAME:root}
7  spring.datasource.password=${SPRING_DATASOURCE_PASSWORD:root}
8  spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
9
10 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5Dialect
11 spring.jpa.hibernate.ddl-auto=${SPRING_JPA_HIBERNATE_DDL_AUTO:update}
12 spring.jpa.show-sql=true
13
14 management.endpoints.web.exposure.include=health,info
15 management.endpoint.health.show-details=always
```

3. Implementación de la API REST

La API expone operaciones principales para la gestión de vuelos bajo la ruta base:

```
/flight
```

Endpoints implementados:

Método	Endpoint	Descripción
POST	/flight/save	Crea un nuevo vuelo.
GET	/flight/listAll	Lista todos los vuelos registrados.
GET	/flight/list/{id}	Consulta un vuelo por identificador.
GET	/flight/topFlights	Consulta vuelos destacados con rating alto y cumplidos.
PUT	/flight	Actualiza un vuelo existente.
DELETE	/flight/{id}	Elimina un vuelo por identificador.

Ejemplo de creación de vuelo:

```
POST /flight/save
```

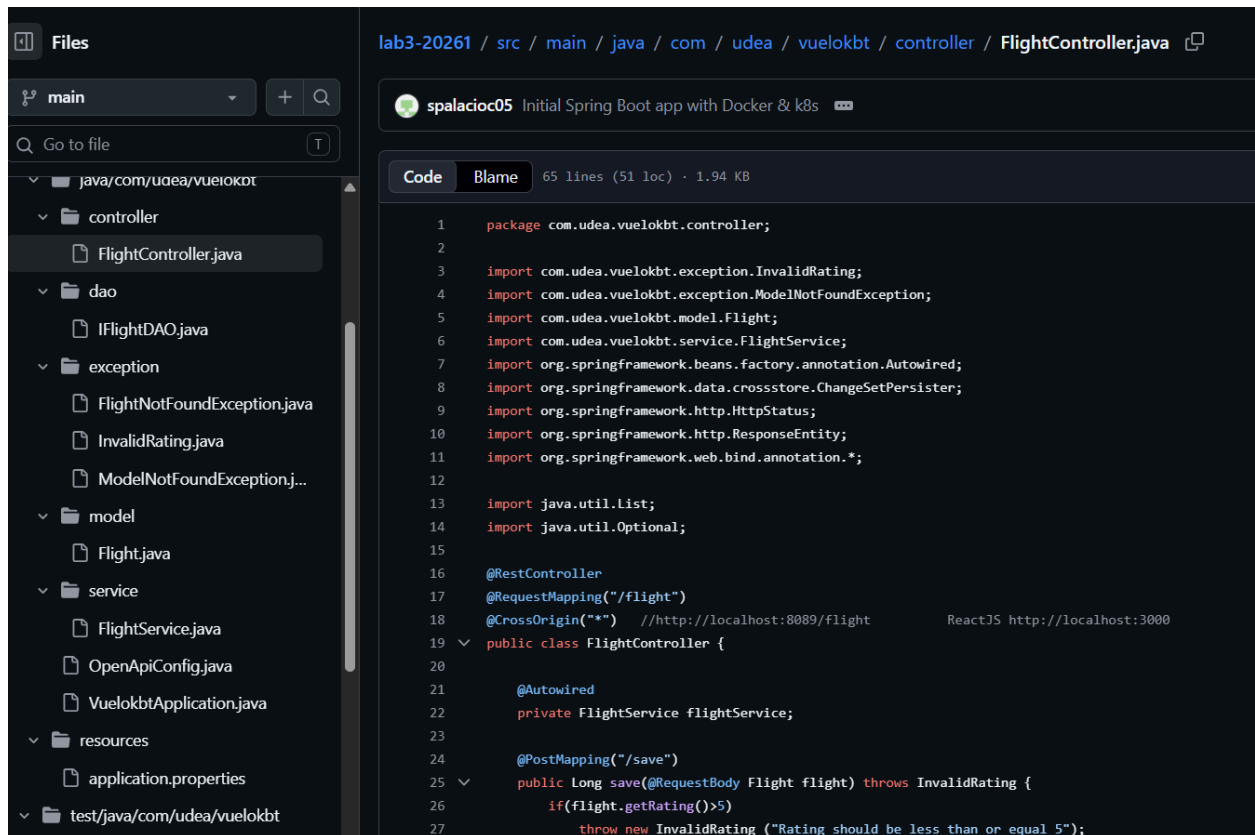
Content-Type: application/json

```
{
  "nombreAvion": "Airbus A320",
  "numeroVuelo": "AV123",
  "origen": "Medellín",
  "destino": "Bogotá",
  "capacidad": 180,
  "rating": 5,
  "planvuelo": 202605220830,
  "cumplido": true
}
```

El repositorio también incluye una consulta JPQL para obtener vuelos destacados:

```
@Query("from Flight f where f.rating>=4 AND f.cumplido=true")
public List<Flight> viewBestFlight();
```

Esta consulta permite filtrar vuelos que tengan una calificación igual o superior a 4 y que hayan sido cumplidos.

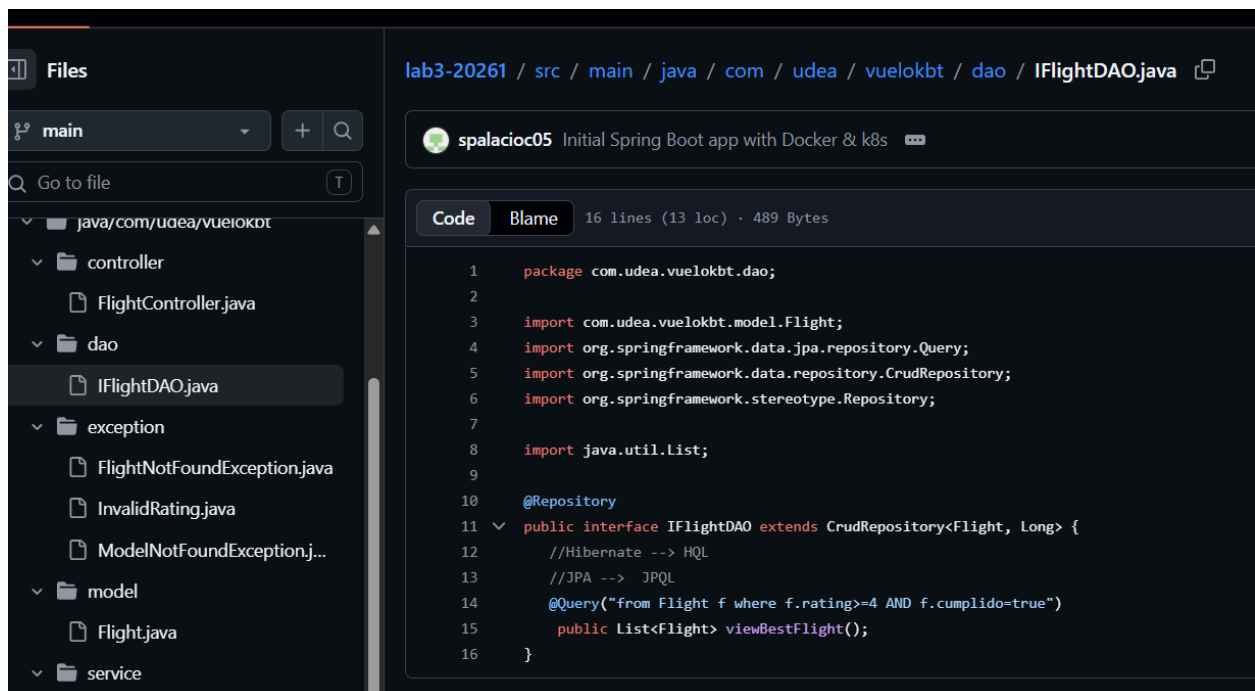


```
lab3-20261 / src / main / java / com / udea / vuelokbt / controller / FlightController.java

spalacioc05 Initial Spring Boot app with Docker & k8s

Code Blame 65 lines (51 loc) · 1.94 KB

1 package com.udea.vuelokbt.controller;
2
3 import com.udea.vuelokbt.exception.InvalidRating;
4 import com.udea.vuelokbt.exception.ModelNotFoundException;
5 import com.udea.vuelokbt.model.Flight;
6 import com.udea.vuelokbt.service.FlightService;
7 import org.springframework.beans.factory.annotation.Autowired;
8 import org.springframework.data.crossstore.ChangeSetPersister;
9 import org.springframework.http.HttpStatus;
10 import org.springframework.http.ResponseEntity;
11 import org.springframework.web.bind.annotation.*;
12
13 import java.util.List;
14 import java.util.Optional;
15
16 @RestController
17 @RequestMapping("/flight")
18 @CrossOrigin("*") //http://localhost:8089/flight ReactJS http://localhost:3000
19 public class FlightController {
20
21     @Autowired
22     private FlightService flightService;
23
24     @PostMapping("/save")
25     public Long save(@RequestBody Flight flight) throws InvalidRating {
26         if(flight.getRating() > 5)
27             throw new InvalidRating ("Rating should be less than or equal 5");
```



```
lab3-20261 / src / main / java / com / udea / vuelokbt / dao / IFlightDAO.java

spalacioc05 Initial Spring Boot app with Docker & k8s

Code Blame 16 lines (13 loc) · 489 Bytes

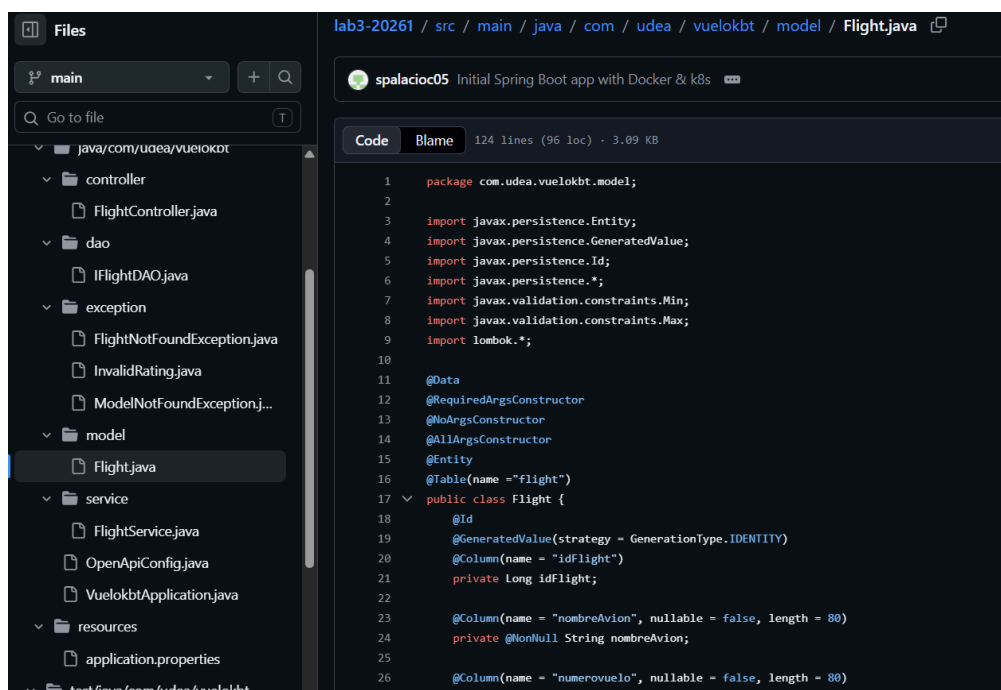
1 package com.udea.vuelokbt.dao;
2
3 import com.udea.vuelokbt.model.Flight;
4 import org.springframework.data.jpa.repository.Query;
5 import org.springframework.data.repository.CrudRepository;
6 import org.springframework.stereotype.Repository;
7
8 import java.util.List;
9
10 @Repository
11 public interface IFlightDAO extends CrudRepository<Flight, Long> {
12     //Hibernate --> HQL
13     //JPA --> JPQL
14     @Query("from Flight f where f.rating >= 4 AND f.cumplido=true")
15     public List<Flight> viewBestFlight();
16 }
```

4. Modelo de datos

La entidad principal del proyecto es Flight, persistida en MySQL.

Campo	Tipo	Descripción
idFlight	Long	Identificador único del vuelo.
nombreAvion	String	Nombre o modelo del avión.
numeroVuelo	String	Código o número del vuelo.
origen	String	Ciudad de origen.
destino	String	Ciudad de destino.
capacidad	int	Número de pasajeros que puede transportar.
rating	int	Calificación asignada al vuelo.
planvuelo	long	Valor numérico asociado al plan de vuelo.
cumplido	Boolean	Indica si el vuelo fue cumplido.

El modelo permite almacenar la información mínima necesaria para realizar operaciones CRUD y consultar vuelos destacados.



The screenshot shows an IDE with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like controller, dao, exception, model, service, and resources. The code editor displays the Flight.java file, which is a Java class representing a flight entity. The code includes package declarations, imports for JPA and validation annotations, and the Flight class itself with various annotations like @Data, @RequiredArgsConstructor, @NoArgsConstructor, @AllArgsConstructor, @Entity, and @Table. The class has private fields for idFlight, nombreAvion, and numeroVuelo, each with corresponding JPA annotations for database mapping.

```
1 package com.udea.vuelokbt.model;
2
3 import javax.persistence.Entity;
4 import javax.persistence.GeneratedValue;
5 import javax.persistence.Id;
6 import javax.persistence.*;
7 import javax.validation.constraints.Min;
8 import javax.validation.constraints.Max;
9 import lombok.*;
10
11 @Data
12 @RequiredArgsConstructor
13 @NoArgsConstructor
14 @AllArgsConstructor
15 @Entity
16 @Table(name = "Flight")
17 public class Flight {
18     @Id
19     @GeneratedValue(strategy = GenerationType.IDENTITY)
20     @Column(name = "idFlight")
21     private Long idFlight;
22
23     @Column(name = "nombreAvion", nullable = false, length = 80)
24     private @NonNull String nombreAvion;
25
26     @Column(name = "numeroVuelo", nullable = false, length = 80)
```

```

Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\spala> $MYSQL_POD = kubectl get pods -l app=mysql -o jsonpath="{.items[0].metadata.name}"
PS C:\Users\spala> echo $MYSQL_POD
mysql-d4c5b6ccd-tjbfz
PS C:\Users\spala> kubectl exec -it $MYSQL_POD -- mysql -uroot -proot -e "SHOW DATABASES; USE vuelokbt; SHOW TABLES; SELECT * FROM flight;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| vuelokbt |
+-----+
+-----+
| Tables_in_vuelokbt |
+-----+
| flight |
+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+
| id_flight | capacidad | cumplido | destino | nombre_avion | numerovuelo | origen | planvuelo | rating |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | 180 | | Bogota | Airbus A320 K8S | K8S123 | Medellin | 202605211830 | 5 |
+-----+-----+-----+-----+-----+-----+-----+-----+
PS C:\Users\spala>

```


5. Documentación con Swagger/OpenAPI

La aplicación incluye configuración OpenAPI mediante `OpenApiConfig.java` y la dependencia `springdoc-openapi-ui`. Esto permite visualizar y probar los endpoints desde una interfaz web.

Ruta esperada de Swagger:

`http://localhost:8089/swagger-ui/index.html`

Swagger facilita la validación manual porque permite revisar los métodos HTTP, rutas, parámetros, cuerpos de solicitud y respuestas esperadas sin necesidad de construir manualmente todas las peticiones desde cero.

The screenshot shows the Swagger UI interface for the 'API de gestión de vuelos' (Flight Management API). The browser address bar shows the URL `http://127.0.0.1:8089/swagger-ui/index.html`. The Swagger logo is in the top left, and the API title 'API de gestión de vuelos' is displayed with version '1.0' and 'OAS3' tags. Below the title, it says 'Documentación de la API de vuelos'. A 'Servers' dropdown menu is set to 'http://127.0.0.1:8089 - Generated server url'. The main section is titled 'flight-controller' and lists six endpoints with their HTTP methods: PUT /flight, POST /flight/save, GET /flight/topFlights, GET /flight/listAll, GET /flight/list/{id}, and DELETE /flight/{id}. Each endpoint is color-coded (orange for PUT, green for POST, blue for GET, and red for DELETE). Below the endpoints, there is a 'Schemas' section with a single entry 'Flight'.

POST

/flight/save

Parameters

No parameters

Request body required

application/json

```
{  "nombreAvion": "Airbus A320",  "numeroVuelo": "AV123",  "origen": "Medellin",  "destino": "Bogota",  "capacidad": 180,  "rating": 5,  "planvuelo": 202605210830,  "cumplido": true}
```

Execute

Responses

Code	Description	Links
200	OK	No links

GET

/flight/topFlights

Parameters

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \  'http://localhost:8089/flight/topFlights' \  -H 'accept: */*'
```

Request URL

```
http://localhost:8089/flight/topFlights
```

Server response

Code	Details
202	Response body <pre>{ "idflight": 1, "nombreAvion": "Airbus A320", "numeroVuelo": "AV123", "origen": "Medellin", "destino": "Bogota", "capacidad": 180, "rating": 5, "planvuelo": 202605210830, "cumplido": true}</pre> Download

Response headers

```
connection: keep-alive  content-type: application/json  date: Thu, 21 May 2026 22:19:36 GMT  keep-alive: timeout=60  transfer-encoding: chunked  vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
```

6. Construcción del proyecto con Maven

Para construir el proyecto se utilizó Maven Wrapper, lo que permite ejecutar Maven sin depender de una instalación global específica.

Comando de empaquetamiento:

```
.\mvnw.cmd -DskipTests package
```

El resultado esperado es la generación del archivo:

```
target/vuelokbt-0.0.1-SNAPSHOT.jar
```

7. Contenerización con Docker

Para empaquetar la aplicación se utilizó un **Dockerfile** basado en Java 11. Este archivo copia el JAR generado por Maven dentro de una imagen y define el comando de arranque.

Dockerfile utilizado:

```
FROM eclipse-temurin:11-jre
COPY target/vuelokbt-0.0.1-SNAPSHOT.jar app.jar

EXPOSE 8089

ENTRYPOINT ["java", "-jar", "app.jar"]
```

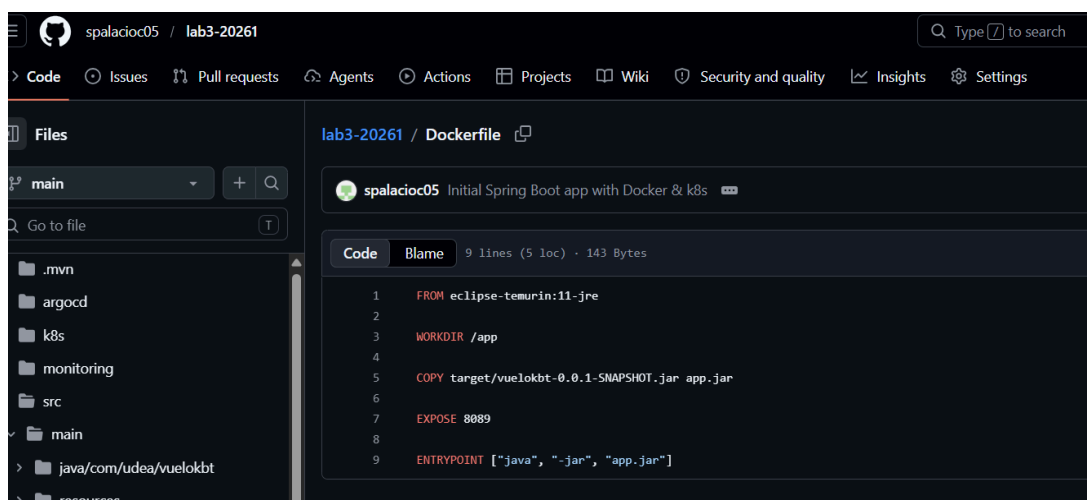
Construcción de imagen:

```
docker build -t vuelokbt-app:latest .
```

Ejecución local con Docker:

```
docker run -p 8089:8089 \
-e SPRING_DATASOURCE_URL=jdbc:mysql://host.docker.internal:3306/vuelokbt \
-e SPRING_DATASOURCE_USERNAME=root \
-e SPRING_DATASOURCE_PASSWORD=root \
vuelokbt-app:latest
```

La imagen Docker permite que la aplicación se ejecute de manera más portable, sin depender directamente del entorno local del desarrollador.



Images [Give feedback](#)

Local My Hub

1.39 GB / 8.05 GB in use 4 images

Last refresh: 1 minute ago

Search

☐	Name	Tag	Image ID	Created	Size	Actions
☐	lisudea/ms-ldap	fix-ldap-entry-20260508-	b7e3ebb8381b	13 days ago	2.07 GB	▶ ⋮ 🗑
☐	lisudea/ms-ldap	fix-ldap-entry-20260508	1ee26dc4f74f	13 days ago	2.07 GB	▶ ⋮ 🗑
☐	gcr.io/k8s-minikube/kicb	v0.0.50	b97074569ae9	3 months ago	1.93 GB	▶ ⋮ 🗑
☐	gcr.io/k8s-minikube/kict	<none>	eb4fec00e8ad	3 months ago	1.93 GB	▶ ⋮ 🗑

Containers [Give feedback](#)

Container CPU usage ⓘ

32.21% / 1200% (12 CPUs available)

Container memory usage ⓘ

654.9MB / 3.59GB

[Show charts](#)

Search



Only show running containers

☐	Name	Container ID	Image	Port(s)	Actions
☐	minikube	45d28b855346	k8s-minikube/kicbase:v0.0.50	52208:22 ↗ Show all ports (5)	▶ ⋮ 🗑

8. Despliegue en Kubernetes con Minikube

Para el despliegue local se utilizó Minikube como clúster Kubernetes. El proyecto incluye manifiestos en la carpeta k8s/, encargados de desplegar MySQL y la aplicación.

Archivos principales:

Archivo	Función
mysql-configmap.yml	Define variables de configuración para MySQL.
mysql-deployment.yml	Despliega MySQL y su Service interno.
app-deployment.yml	Despliega la aplicación Spring Boot y su Service.

Flujo de despliegue recomendado:

```
minikube start --driver=docker
minikube docker-env --shell powershell | Invoke-Expression

.\mvnw.cmd -DskipTests package
docker build -t vuelokbt-app:latest .

kubectl apply -f .\k8s\mysql-configmap.yml
kubectl apply -f .\k8s\mysql-deployment.yml
kubectl apply -f .\k8s\app-deployment.yml

kubectl get pods
kubectl get svc
minikube service vuelokbt-service --url
```

La aplicación se conecta a MySQL usando el nombre del Service dentro del clúster:

```
mysql
```

Esto permite que los pods de la aplicación no dependan de una dirección IP fija, sino de la resolución interna de Kubernetes.

```
Administrador: Windows Pow x + v
PS C:\Users\spala> cd "C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261"
PS C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261> minikube -p minikube docker-env
--shell powershell | Invoke-Expression
PS C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261> docker info | Select-String "Name"

Name: minikube

PS C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261>
```

```
Administrador: Windows Pow x + v
=> => sha256:945a23ebc056857af5d3da26e9b6313dccc6a75bcec498085c4d773da0dea480 20.17MB / 20.17MB 45.9s
=> => sha256:8e64f7ea0147323b223160b0fa39bd3aa9e163f03bf8de6a37e1dc7e9c4009ef 2.13kB / 2.13kB 0.0s
=> => sha256:8ecaff8fcded8f37b5d036e45c9b200392e08e25b32788e5d4ec94f2a8e20280 9.00kB / 9.00kB 0.0s
=> => sha256:6f5c5aa4e145204b113f983c003ff8ad6489394294ef95ec030bc94e3daded54 41.55MB / 41.55MB 78.9s
=> => sha256:d1f60238b9dcba29793d51390d5e7e5c965d5ffea5df15766f2e9f60e21433bc 47.34MB / 47.34MB 93.7s
=> => sha256:5eb7a53060679f15918daef9dc22422d2c1914f40d1c4a21bcd0e3b5896b71a0 160B / 160B 46.8s
=> => sha256:11d6883999e9e9f4717faf02f3fb28737ac731d5f8c6eb06ed34764cf17d2196 2.28kB / 2.28kB 46.9s
=> => extracting sha256:6f5c5aa4e145204b113f983c003ff8ad6489394294ef95ec030bc94e3daded54 7.6s
=> => extracting sha256:1c24335ddd46023ff99bd665bd8ea6798464f7bbf501718edcf2eb4696e5f408 0.0s
=> => extracting sha256:945a23ebc056857af5d3da26e9b6313dccc6a75bcec498085c4d773da0dea480 1.9s
=> => extracting sha256:d1f60238b9dcba29793d51390d5e7e5c965d5ffea5df15766f2e9f60e21433bc 2.4s
=> => extracting sha256:5eb7a53060679f15918daef9dc22422d2c1914f40d1c4a21bcd0e3b5896b71a0 0.0s
=> => extracting sha256:11d6883999e9e9f4717faf02f3fb28737ac731d5f8c6eb06ed34764cf17d2196 0.0s
=> [internal] load build context
=> transferring context: 49.67MB 3.6s
=> [2/3] WORKDIR /app 3.5s
=> [3/3] COPY target/vuelokbt-0.0.1-SNAPSHOT.jar app.jar 0.3s
=> exporting to image 0.3s
=> exporting layers 0.2s
=> writing image sha256:50f1e56475522cfff47f04b7a3738032c84332722f30e539ecb33db0d65a1468 0.1s
=> naming to docker.io/library/vuelokbt-app:latest 0.0s

View build details: docker-desktop://dashboard/build/default/default/3ql8cdiwiugk4f7qz6t62q23m
PS C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261> docker images | Select-String "vuelokbt-app"

vuelokbt-app latest 50f1e5647552 21 seconds ago 344MB

PS C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261>
```

```
Administrador: Windows Pow x + v
S C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261> kubectl apply -f k8s/mysql-configmap.yml
onfigmap/mysql-config created
S C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261> kubectl apply -f k8s/mysql-deployment.yml
eployment.apps/mysql created
ervice/mysql created
S C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261> kubectl apply -f k8s/app-deployment.yml
eployment.apps/vuelokbt-app created
ervice/vuelokbt-service created
S C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261> |
```

```
PS C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261> kubectl get pods
NAME READY STATUS RESTARTS AGE
mysql-d4c5b6ccd-tjbfz 1/1 Running 0 3m25s
vuelokbt-app-57f54cfbdf-6h9b5 1/1 Running 2 (35s ago) 3m23s
vuelokbt-app-57f54cfbdf-z7x5m 1/1 Running 2 (35s ago) 3m23s
PS C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261>
```

[illegible]

The screenshot shows a Windows PowerShell terminal window with the title "Administrador: Windows Pow". The command entered is `minikube service vuelokbt-service --url http://127.0.0.1:61125`. Below the command, a red exclamation mark icon is followed by the text: "Porque estás usando controlador Docker en windows, la terminal debe abrirse para ejecutarlo."

HAL ExplorerThemeSettingsAbout

Edit Headers /

Go!

Links

Relation	Name	Title	HTTP Request	Doc
flights			< > + - > > x	
profile			< > + - > > x	

Response Status

200 (OK)

Response Headers

connection	keep-alive
content-type	application/hal+json
date	Thu, 21 May 2025 23:31:35 GMT
keep-alive	timeout=60
transfer-encoding	chunked
vary	Origin, Access-Control-Request-Method, Access-Control-Request-Headers

Response Body

```
{
  "_links": {
    "flights": {
      "href": "http://127.0.0.1:61125/flights"
    },
    "profile": {
      "href": "http://127.0.0.1:61125/profile"
    }
  }
}
```


9. Verificación de MySQL en Kubernetes

Para comprobar que la base de datos se encuentra activa dentro del clúster, se puede ingresar al contenedor MySQL o ejecutar comandos directamente con `kubectl exec`.

Ejemplo:

```
kubectl exec -it deployment/mysql -- mysql -uroot -proot -e "SHOW DATABASES;"
```

Consultas:

```
SHOW DATABASES;  
USE vuelokbt;  
SHOW TABLES;  
SELECT * FROM flight;
```

Estas consultas permiten evidenciar que la base de datos existe, que la tabla fue creada y que los datos registrados desde la API realmente se están persistiendo.

10. Sincronización GitOps con ArgoCD

El laboratorio también incorpora una estrategia GitOps usando ArgoCD. Para esto, el proyecto incluye el archivo:

argocd/application.yml

Este manifiesto define una aplicación de ArgoCD que apunta al repositorio GitHub y a la carpeta **k8s/**, donde se encuentran los recursos Kubernetes.

Configuración principal:

Propiedad	Valor
Nombre de la aplicación	vuelokbt-gitops
Namespace	argocd
Repositorio	https://github.com/spalacioc05/lab3-20261.git
Rama	main
Ruta de manifiestos	k8s
Namespace destino	default
Sincronización automática	Habilitada
Self-heal	Habilitado
Prune	Habilitado

Comando para aplicar:

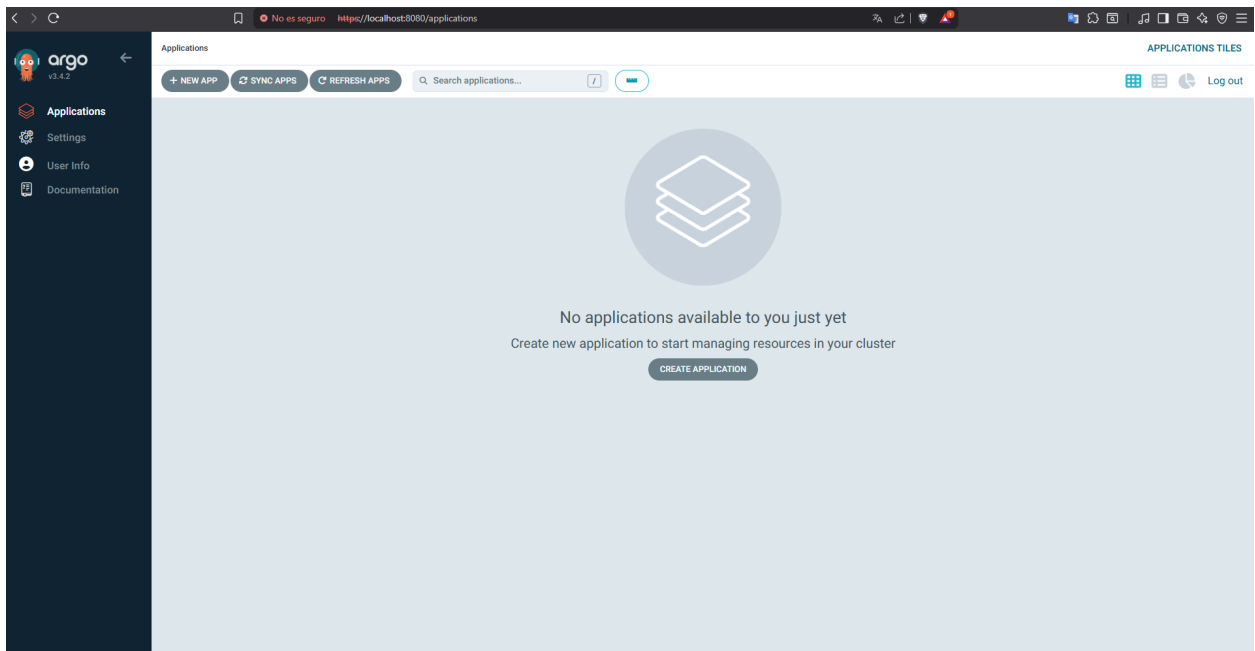
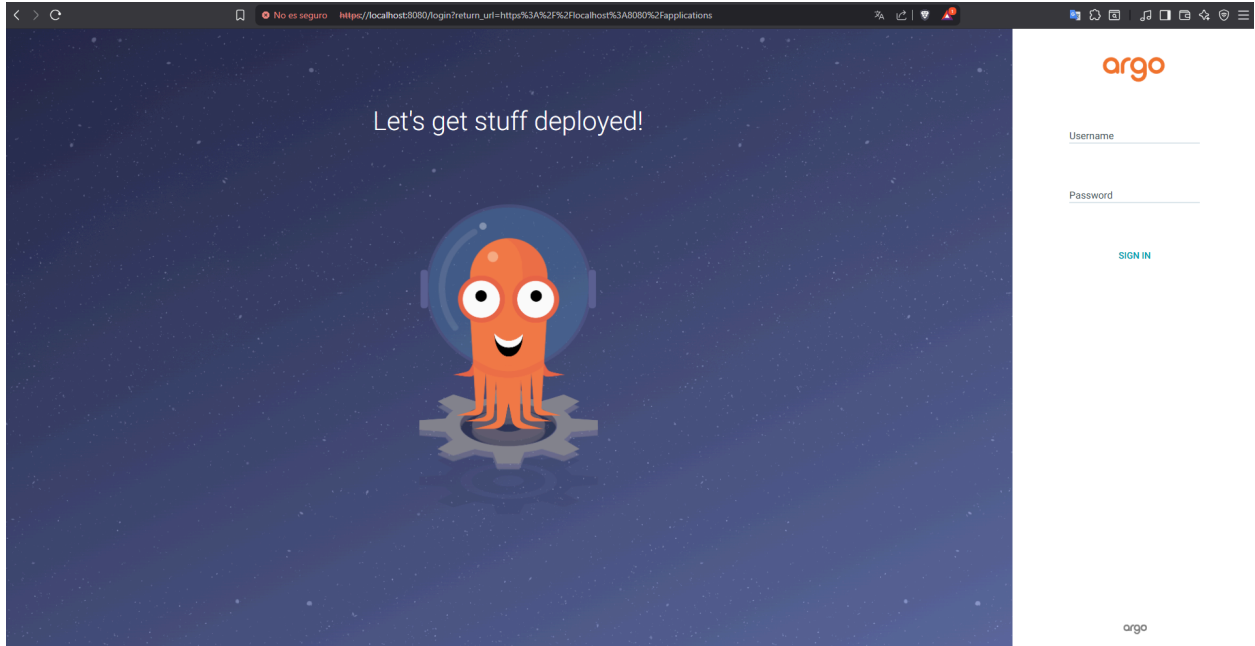
```
kubectl apply -f .\argocd\application.yml -n argocd
```

Con esta configuración, ArgoCD puede detectar cambios en los manifiestos del repositorio y sincronizarlos con el clúster. Esto refleja un enfoque GitOps, donde Git se convierte en la fuente de verdad para la infraestructura declarativa.

```

deployment.apps/argocd-applicationset-controller serverside-applied
deployment.apps/argocd-dex-server serverside-applied
deployment.apps/argocd-notifications-controller serverside-applied
deployment.apps/argocd-redis serverside-applied
deployment.apps/argocd-repo-server serverside-applied
deployment.apps/argocd-server serverside-applied
statefulset.apps/argocd-application-controller serverside-applied
networkpolicy.networking.k8s.io/argocd-application-controller-network-policy serverside-applied
networkpolicy.networking.k8s.io/argocd-applicationset-controller-network-policy serverside-applied
networkpolicy.networking.k8s.io/argocd-dex-server-network-policy serverside-applied
networkpolicy.networking.k8s.io/argocd-notifications-controller-network-policy serverside-applied
networkpolicy.networking.k8s.io/argocd-redis-network-policy serverside-applied
networkpolicy.networking.k8s.io/argocd-repo-server-network-policy serverside-applied
networkpolicy.networking.k8s.io/argocd-server-network-policy serverside-applied
PS C:\Users\spala> kubectl get pods -n argocd
NAME                                     READY   STATUS             RESTARTS   AGE
argocd-application-controller-0         0/1     ContainerCreating   0           5s
argocd-applicationset-controller-bc84c94bc-9w9z6  0/1     ContainerCreating   0           9s
argocd-dex-server-598547cb8-mxmv6       0/1     Init:0/1            0           9s
argocd-notifications-controller-5f59d79f8c-8jtw5  0/1     ContainerCreating   0           9s
argocd-redis-8b5b5b56d-rf29d           0/1     Init:0/1            0           9s
argocd-repo-server-ff5578b69-l2grw      0/1     Init:0/1            0           9s
argocd-server-5f898b9cbc-7rw6g         0/1     ContainerCreating   0           8s
PS C:\Users\spala> kubectl wait --for=condition=Ready pod --all -n argocd --timeout=300s
pod/argocd-application-controller-0 condition met
pod/argocd-applicationset-controller-bc84c94bc-9w9z6 condition met
pod/argocd-dex-server-598547cb8-mxmv6 condition met
pod/argocd-notifications-controller-5f59d79f8c-8jtw5 condition met
pod/argocd-redis-8b5b5b56d-rf29d condition met
pod/argocd-repo-server-ff5578b69-l2grw condition met
pod/argocd-server-5f898b9cbc-7rw6g condition met
PS C:\Users\spala> kubectl get pods -n argocd
NAME                                     READY   STATUS    RESTARTS   AGE
argocd-application-controller-0         1/1     Running   0           2m24s
argocd-applicationset-controller-bc84c94bc-9w9z6  1/1     Running   0           2m28s
argocd-dex-server-598547cb8-mxmv6       1/1     Running   0           2m28s
argocd-notifications-controller-5f59d79f8c-8jtw5  1/1     Running   0           2m28s
argocd-redis-8b5b5b56d-rf29d           1/1     Running   0           2m28s
argocd-repo-server-ff5578b69-l2grw      1/1     Running   0           2m28s
argocd-server-5f898b9cbc-7rw6g         1/1     Running   0           2m27s
PS C:\Users\spala>

```



```
1  apiVersion: argoproj.io/v1alpha1    Unknown property 'apiVersion'
2  kind: Application                    Unknown property 'kind'
3  metadata:                          Unknown property 'metadata'
4    name: vuelokbt-gitops
5    namespace: argocd
6  spec:                              Unknown property 'spec'
7    project: default
8    source:
9      repoURL: https://github.com/spalacioc05/lab3-20261.git
10     targetRevision: main
11     path: k8s
12   destination:
13     server: https://kubernetes.default.svc
14     namespace: default
15   syncPolicy:
16     automated:
17       prune: true
18       selfHeal: true
19     syncOptions:
20       - CreateNamespace=true
```

```
PS C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261> kubectl apply -f argocd/application.yml
application.argoproj.io/vuelokbt-gitops created
PS C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261> kubectl get applications -n argocd
NAME          SYNC STATUS  HEALTH STATUS
vuelokbt-gitops
PS C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261> 
```

11. Escalabilidad horizontal

La escalabilidad horizontal se trabajó aumentando el número de réplicas del Deployment de la aplicación. En la documentación del proyecto se indica que el Deployment vuelokbt-app fue ajustado a **4 réplicas**.

Comandos de verificación:

```
kubectl get deployment vuelokbt-app
kubectl get pods -l app=vuelokbt
```

Escalado manual adicional:

```
kubectl scale deployment/vuelokbt-app --replicas=6
kubectl get pods -l app=vuelokbt
```

Este ejercicio permite observar cómo Kubernetes crea múltiples pods de la aplicación y distribuye el tráfico a través del Service correspondiente.

```
PS C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261> kubectl get svc
NAME                TYPE          CLUSTER-IP      EXTERNAL-IP  PORT(S)          AGE
kubernetes           ClusterIP     10.96.0.1        <none>        443/TCP          114m
mysql                ClusterIP     10.109.146.42    <none>        3306/TCP          90m
vuelokbt-service     LoadBalancer 10.98.210.117    127.0.0.1    8089:31134/TCP    90m
PS C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261> kubectl get applications -n argocd
NAME                SYNC STATUS  HEALTH STATUS
vuelokbt-gitops     Synced       Healthy
PS C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261>
```

Applications

+ NEW APP SYNC APPS REFRESH APPS Search applications...

vuelokbt-gitops

Project: default

Labels:

Status: Healthy Synced

Repository: https://github.com/spalacioc05/lab3-2...

Target R...: main

Path: k8s

Destinat...: in-cluster

Namesp...: default

Created ...: 05/21/2026 19:50:03 (6 minutes ago)

Last Sync: 05/21/2026 19:50:30 (5 minutes ago)

SYNC REFRESH DELETE

Application filters

☐ ★ Favorites Only

SYNC STATUS

☐ ⏸ Unknown 0

☐ ✅ Synced 1

☐ ⚠ OutOfSync 0

HEALTH STATUS

☐ 🔄 Progressing 0

☐ ⏸ Suspended 0

☐ ❤️ Healthy 1

☐ 💔 Degraded 0

☐ 🧑 Missing 0

☐ ? Unknown 0

OPERATION STATUS

☐ 🔄 Syncing 0

☐ ✅ Sync OK 1

☐ ❗ Sync error 0

☐ ❌ Sync failed 0

```
PS C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261> kubectl get deployment vuelokbt-app
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
vuelokbt-app  4/4     4             4           96m
PS C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261> kubectl get pods -l app=vuelokbt
NAME                                READY   STATUS    RESTARTS   AGE
vuelokbt-app-57f54cfbdf-6h9b5      1/1     Running   3 (91m ago)  96m
vuelokbt-app-57f54cfbdf-jmqrk      1/1     Running   0           91s
vuelokbt-app-57f54cfbdf-whgh6      1/1     Running   0           91s
vuelokbt-app-57f54cfbdf-z7r5m      1/1     Running   3 (91m ago)  96m
PS C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261> kubectl get applications -n argocd
NAME          SYNC STATUS   HEALTH STATUS
vuelokbt-gitops Synced        Healthy
PS C:\Users\spala\Desktop\UDEA\2026-1\ARQUITECTURA DE SOFTWARE\LABORATORIOS\lab3-20261>
```

```
PS C:\Users\spala> 1..10 | ForEach-Object {  
    >> kubectl exec deployment/vuelokbt-app -- curl -s -o /dev/null -w "REQ=%$_ HTTP=%{http_code} TIME=%{time_total}"'n" http://vuelokbt-service:8089/flight/lis  
    >> }  
All  
REQ=1 HTTP=200 TIME=14.279529  
REQ=2 HTTP=200 TIME=0.985655  
REQ=3 HTTP=200 TIME=0.029172  
REQ=4 HTTP=200 TIME=0.925940  
REQ=5 HTTP=200 TIME=1.161807  
REQ=6 HTTP=200 TIME=0.029879  
REQ=7 HTTP=200 TIME=0.017459  
REQ=8 HTTP=200 TIME=0.026625  
REQ=9 HTTP=200 TIME=0.033927  
REQ=10 HTTP=200 TIME=0.077721  
PS C:\Users\spala>
```

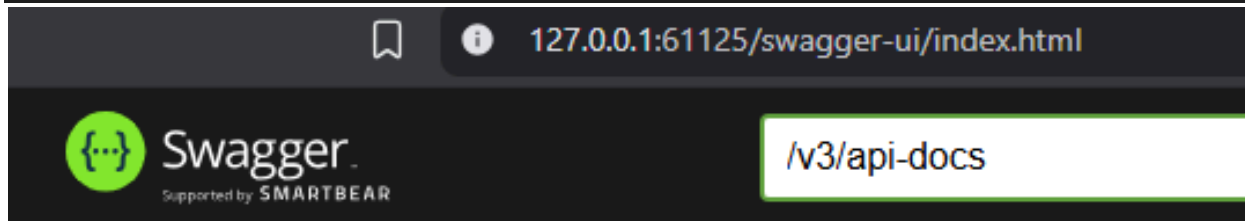

12. Pruebas funcionales

Las pruebas funcionales pueden realizarse desde Swagger o Postman. A continuación se presentan los casos básicos que permiten comprobar el comportamiento principal de la API.

Crear vuelo

```
POST /flight/save
Content-Type: application/json
```

```
{
  "nombreAvion": "Boeing 737",
  "numeroVuelo": "LA456",
  "origen": "Bogotá",
  "destino": "Cali",
  "capacidad": 160,
  "rating": 4,
  "planvuelo": 202605221030,
  "cumplido": true
}
```



API de gestión de vuelos 1.0 OAS3

/v3/api-docs

Documentación de la API de vuelos

Servers

http://127.0.0.1:61125 - Generated server url

flight-controller

PUT /flight

POST /flight/save

GET /flight/topFlights

GET /flight/listAll

GET /flight/list/{id}

DELETE /flight/{id}

Schemas

Flight >

POST /flight/save

Parameters

Cancel

Reset

No parameters

Request body required

application/json

```
{
  "nombreAvion": "Airbus A320",
  "numeroVuelo": "AV123",
  "origen": "Medellin",
  "destino": "Bogota",
  "capacidad": 180,
  "rating": 5,
  "planVuelo": 202605210830,
  "cumplido": true
}
```

Execute

Responses

Listar vuelos

GET /flight/listAll

The screenshot shows a REST client interface with the following sections:

- Parameters:** A tab with a "Cancel" button. Below it, it says "No parameters". At the bottom of this section are "Execute" and "Clear" buttons.
- Responses:** A section containing:
 - Curl:** A code block with the command:

```
curl -X 'GET' \
'http://localhost:8089/flight/listAll' \
-H 'accept: */*'
```
 - Request URL:** A text field containing `http://localhost:8089/flight/listAll`.
 - Server response:** A section with a "Details" button.
 - Code:** A table with one row:

Code	Details
200	Response body <pre>{ "idFlight": 1, "nombreAvion": "Airbus A320", "numeroVuelo": "AV123", "origen": "Medellin", "destino": "Bogota", "capacidad": 180, "rating": 5, "planVuelo": "202605210830", "cumplido": true }</pre> Response headers <pre>connection: keep-alive content-type: application/json date: Thu, 21 May 2026 22:17:40 GMT keep-alive: timeout=60 transfer-encoding: chunked vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers</pre>

Consultar vuelo por ID

GET /flight/list/1

The screenshot shows a REST client interface with the following sections:

- Parameters:** A tab with a "Cancel" button. Below it, a table lists parameters:

Name	Description
id * required	
integer(\$int64)	
(path)	

A text input field contains the value "1". At the bottom of this section are "Execute" and "Clear" buttons.
- Responses:** A section containing:
 - Curl:** A code block with the command:

```
curl -X 'GET' \
'http://localhost:8089/flight/list/1' \
-H 'accept: */*'
```
 - Request URL:** A text field containing `http://localhost:8089/flight/list/1`.
 - Server response:** A section with a "Details" button.
 - Code:** A table with one row:

Code	Details
200	Response body <pre>{ "idFlight": 1, "nombreAvion": "Airbus A320", "numeroVuelo": "AV123", "origen": "Medellin", "destino": "Bogota", "capacidad": 180, "rating": 5, "planVuelo": "202605210830", "cumplido": true }</pre>

Consultar mejores vuelos

GET /flight/topFlights

The screenshot shows a REST client interface with the following sections:

- Parameters:** A tab labeled "Parameters" with a "Cancel" button. Below it, it says "No parameters".
- Execute:** A blue button labeled "Execute" and a grey button labeled "Clear".
- Responses:** A section containing:
 - Curl:** A code block with the command:

```
curl -X 'GET' \
'http://localhost:8089/flight/topFlights' \
-H 'accept: */*'
```
 - Request URL:** A text field containing `http://localhost:8089/flight/topFlights`.
 - Server response:** A section with a "Code" column showing "202" and a "Details" column showing "Undocumented".
 - Response body:** A large text area containing a JSON object:

```
{
  "idFlight": 1,
  "nombreAvion": "Airbus A320",
  "numeroVuelo": "AV123",
  "origen": "Medellin",
  "destino": "Bogota",
  "capacidad": 180,
  "rating": 5,
  "planvuelo": 202605210830,
  "cumplido": true
}
```
 - Response headers:** A text area containing:

```
connection: keep-alive
content-type: application/json
date: Thu, 21 May 2026 22:19:36 GMT
keep-alive: timeout=60
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
```

Actualizar vuelo

PUT /flight

Content-Type: application/json

```
{
  "idFlight": 1,
  "nombreAvion": "Airbus A321",
  "numeroVuelo": "AV123",
  "origen": "Medellín",
  "destino": "Cartagena",
  "capacidad": 185,
  "rating": 5,
  "planvuelo": 202605221100,
  "cumplido": true
}
```

PUT /flight

Parameters

Cancel

Reset

No parameters

Request body required

application/json

```
{
  "idflight": 1,
  "nombreavion": "Boeing 737",
  "numerovuelo": "AV456",
  "origen": "Medellin",
  "destino": "Cartagena",
  "capacidad": 150,
  "rating": 4,
  "planvuelo": 202605211030,
  "cumplido": false
}
```

Execute

Clear

Responses

Curl

```
curl -X 'PUT' \
'http://localhost:8089/flight' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
  "idflight": 1,
  "nombreavion": "Boeing 737",
  "numerovuelo": "AV456",
  "origen": "Medellin",
  "destino": "Cartagena",
  "capacidad": 150,
  "rating": 4,
  "planvuelo": 202605211030,
  "cumplido": false
}
```

Conclusiones

El laboratorio permitió integrar varias piezas importantes de una arquitectura backend moderna. Se partió de una aplicación Spring Boot para gestionar vuelos y se avanzó hacia un escenario más completo, donde la aplicación se documenta, se empaqueta, se despliega y se administra mediante herramientas de contenedores y orquestación.

Uno de los aprendizajes más importantes fue comprender que una API no termina cuando los endpoints funcionan localmente. Para que un servicio sea realmente desplegable, debe tener configuración flexible, conexión adecuada a base de datos, empaquetamiento reproducible, manifiestos de infraestructura y evidencia clara de ejecución.

También se evidenció la importancia de separar las responsabilidades. Spring Boot se encarga de la lógica de aplicación; MySQL de la persistencia; Docker del empaquetamiento; Kubernetes de la ejecución y escalabilidad; y ArgoCD de la sincronización declarativa desde Git. Cada herramienta cumple una función específica dentro de la arquitectura.

El trabajo con Kubernetes y Minikube permitió observar cómo la aplicación deja de ejecutarse como un proceso local y pasa a funcionar como un conjunto de pods, services y deployments. Esto ayuda a entender conceptos como réplica, balanceo interno, service discovery y despliegue declarativo.

Finalmente, el laboratorio deja una base funcional para seguir mejorando. Entre las mejoras futuras se encuentran fortalecer el manejo de errores, ampliar pruebas automatizadas, implementar HATEOAS de forma más explícita en las respuestas, completar monitoreo con Prometheus/Grafana y mejorar la documentación operativa del proyecto.

Bibliografía

- Spring Boot. Documentación oficial: <https://spring.io/projects/spring-boot>
- Spring Data JPA. Documentación oficial: <https://spring.io/projects/spring-data-jpa>
- Spring HATEOAS. Documentación oficial: <https://spring.io/projects/spring-hateoas>
- Springdoc OpenAPI. Documentación oficial: <https://springdoc.org/>
- MySQL. Documentación oficial: <https://dev.mysql.com/doc/>
- Docker. Documentación oficial: <https://docs.docker.com/>
- Kubernetes. Documentación oficial: <https://kubernetes.io/docs/>
- Minikube. Documentación oficial: <https://minikube.sigs.k8s.io/docs/>
- ArgoCD. Documentación oficial: <https://argo-cd.readthedocs.io/>
- Grafana. Documentación oficial: <https://grafana.com/docs/>

Proyecto anexo en GitHub

El proyecto completo se encuentra disponible en el siguiente repositorio:

<https://github.com/spalacioc05/lab3-20261.git>

El repositorio incluye el código fuente de la aplicación Spring Boot, configuración Maven, Dockerfile, manifiestos Kubernetes, configuración de ArgoCD, monitoreo opcional con Grafana y documentación del laboratorio.

